

**Федеральное государственное автономное образовательное
учреждение высшего образования «Национальный
исследовательский университет ИТМО»**

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №4
По Основам Программной Инженерии
Вариант 55147

Выполнил:

Ларионов Владислав Васильевич

Группа Р3209

Проверил:

Ермаков Михаил Константинович

Санкт-Петербург, 2026

Содержание

Задание	3
Выполнение задания №2.....	4
Выполнение задания №3.....	6
Выполнение задания №4.....	9
Программа	15
Вывод:	15

Задание

1. Для своей программы из [лабораторной работы #3](#) по дисциплине "Веб-программирование" реализовать:

- MBean, считающий общее число установленных пользователем точек, а также число точек, не попадающих в область. В случае, если координаты установленной пользователем точки вышли за пределы отображаемой области координатной плоскости, разработанный MBean должен отправлять оповещение об этом событии.
- MBean, определяющий площадь получившейся фигуры.

2. С помощью утилиты JConsole провести мониторинг программы:

- Снять показания MBean-классов, разработанных в ходе выполнения задания 1.
- Определить имя и версию ОС, под управлением которой работает JVM.

3. С помощью утилиты VisualVM провести мониторинг и профилирование программы:

- Снять график изменения показаний MBean-классов, разработанных в ходе выполнения задания 1, с течением времени.
- Определить имя класса, объекты которого занимают наибольший объём памяти JVM; определить пользовательский класс, в экземплярах которого находятся эти объекты.

4. С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в [программе](#). По результатам локализации и устранения проблемы необходимо составить отчёт, в котором должна содержаться следующая информация:

- Описание выявленной проблемы.
- Описание путей устранения выявленной проблемы.
- Подробное (со скриншотами) описание алгоритма действий, который позволил выявить и локализовать проблему.

Студент должен обеспечить возможность воспроизведения процесса поиска и локализации проблемы по требованию преподавателя.

Выполнение задания №2

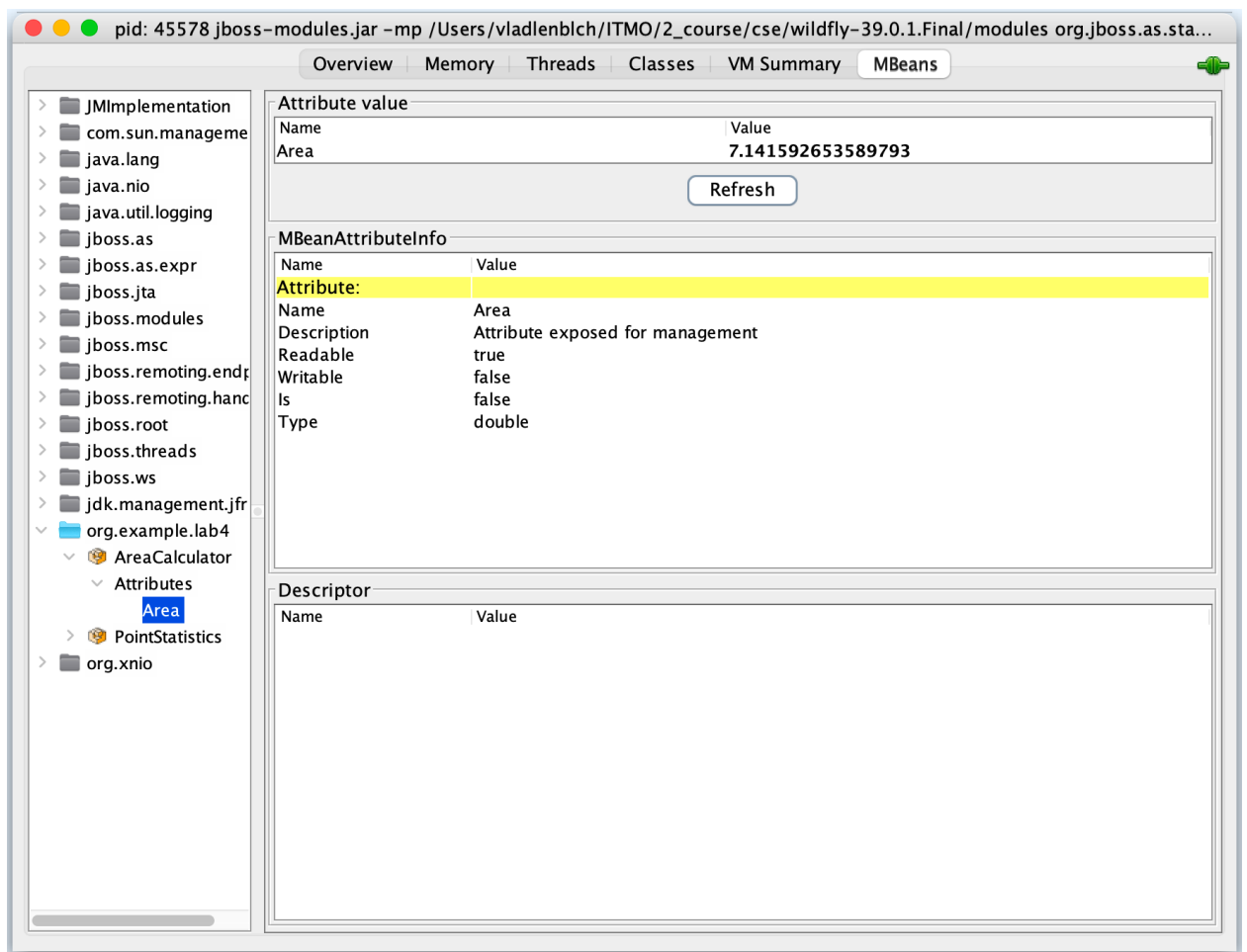


Рис. 2.1 – пример снятых показаний AreaCalculator

Далее скриншоты конкретно снятых показаний (без интерфейса JConsole)

Attribute value	
Name	Value
TotalPoints	9
Refresh	

Рис. 2.2 – пример снятых показаний PointStatistics для TotalPoints

Attribute value	
Name	Value
MissPoints	3
Refresh	

Рис. 2.3 – пример снятых показаний PointStatistics для MissPoints

Attribute value	
Name	Value
Name	Mac OS X

Рис. 2.3 – снятые показания имени ОС, под управлением которой работает JVM

Attribute value	
Name	Value
Version	26.3.1

Рис. 2.4 – снятые показания версии ОС, под управлением которой работает JVM

The screenshot shows the JBoss IDE interface with the 'Notification buffer' tab selected. The buffer contains three entries, each representing a notification event. The first entry is highlighted in blue.

TimeStamp	Type	UserData	SeqNum	Message	Event	Source
20:13:03...	org.examp...		3	Point is out...	javax.man...	org.example.la...
20:12:56...	org.examp...		2	Point is out...	javax.man...	org.example.la...
20:12:24...	org.examp...		1	Point is out...	javax.man...	org.example.la...

At the bottom of the notification buffer, there are three buttons: 'Subscribe', 'Unsubscribe', and 'Clear'.

Рис. 2.5 – снятые оповещения о том, что точка не видна на графике

Выполнение задания №3

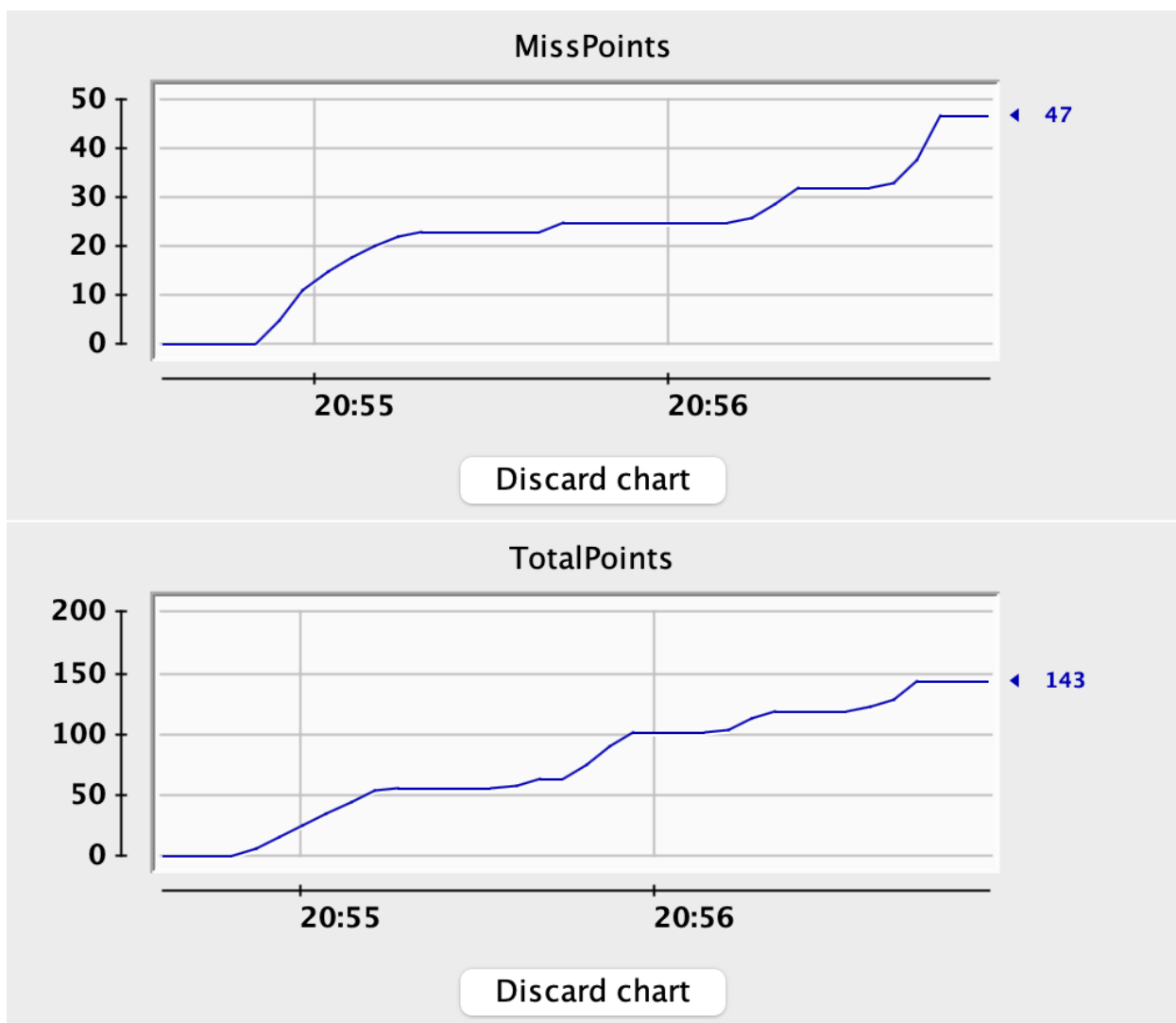


Рис. 3.1 – графики снятых показателей MBean-классов с течением времени

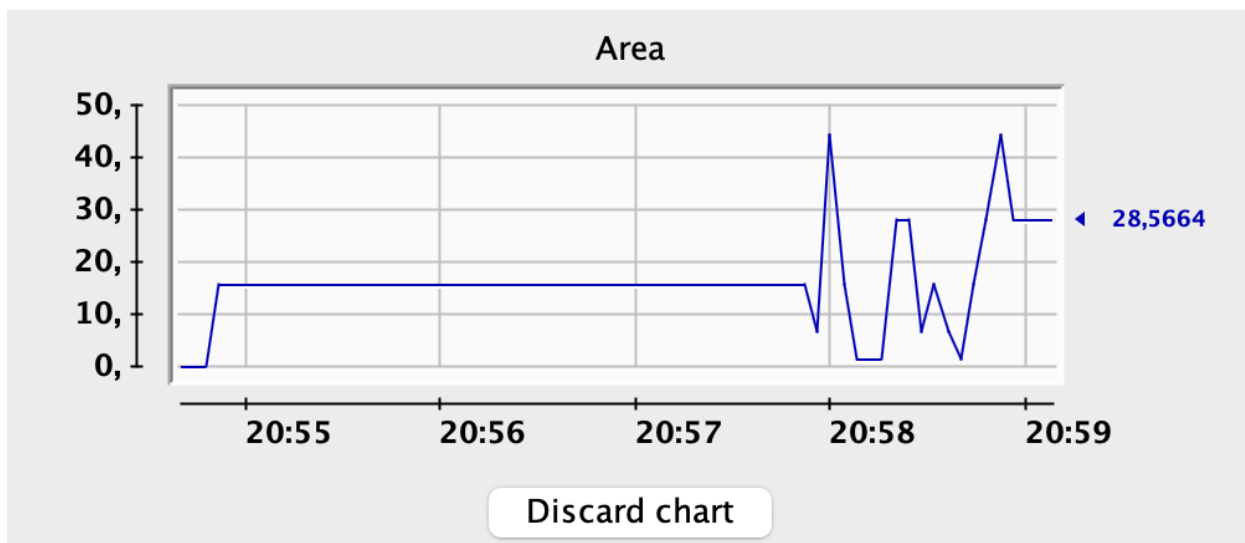


Рис. 3.2 – график снятых показателей MBean-классов с течением времени

Profiler Snapshot			
Name	Live Bytes		Live Objects
byte[]	32 851 672 B (24,8 %)		340 142 (11,1 %)
java.util.HashMap\$Node	9 664 864 B (7,3 %)		302 027 (9,8 %)
java.lang.Object[]	7 901 600 B (6 %)		243 980 (7,9 %)
java.lang.String	6 608 616 B (5 %)		275 359 (9 %)
java.util.HashMap\$Node[]	6 278 696 B (4,7 %)		75 259 (2,4 %)
int[]	4 816 160 B (3,6 %)		40 241 (1,3 %)
java.util.HashMap	4 615 392 B (3,5 %)		96 154 (3,1 %)
java.util.HashMap\$KeyIterator	4 104 800 B (3,1 %)		102 620 (3,3 %)
char[]	2 863 960 B (2,2 %)		58 672 (1,9 %)
java.lang.Class	2 527 376 B (1,9 %)		21 072 (0,7 %)
java.lang.reflect.Type[]	1 601 752 B (1,2 %)		74 583 (2,4 %)
java.lang.reflect.Method	1 587 168 B (1,2 %)		18 036 (0,6 %)
java.nio.HeapCharBuffer	1 578 920 B (1,2 %)		28 195 (0,9 %)
java.util.ArrayList	1 532 088 B (1,2 %)		63 837 (2,1 %)
java.util.HashMap\$EntryIterator	1 369 640 B (1 %)		34 241 (1,1 %)
java.util.TreeMap\$Entry	1 326 040 B (1 %)		33 151 (1,1 %)
java.lang.reflect.Parameter	1 203 240 B (0,9 %)		30 081 (1 %)

Рис. 3.3 – снимок результата распределения памяти

Heap Dump			
Name	Count	Size	Retained (sort to get)
byte[]	222 394 (14,5 %)	25 026 776 B (34,7 %)	n/a
byte[]#56935 : 789 010 items		789 032 B (1,1 %)	n/a
<items>			
<references>			
cen in java.util.zip.ZipFile\$Source#414		80 B (0 %)	n/a
byte[]#48408 : 782 745 items		782 768 B (1,1 %)	n/a
<items>			
<references>			
cen in java.util.zip.ZipFile\$Source#328		80 B (0 %)	n/a
value in java.util.HashMap\$Node#49186		32 B (0 %)	n/a
zsrc in java.util.zip.ZipFile\$CleanableResource#719		32 B (0 %)	n/a
zsrc in java.util.zip.ZipFile\$CleanableResource#720		32 B (0 %)	n/a
byte[]#163461 : 680 384 items		680 400 B (0,9 %)	n/a

Рис. 3.4 – распределение памяти по классу byte[]

Heap Dump			
Name	Count	Size	Retained (sort to get)
org.example.mbeans.PointStatistics	2 (0 %)	80 B (0 %)	n/a
org.example.service.DatabaseService\$Proxy\$_\$\$_WeldClientProxy	1 (0 %)	48 B (0 %)	n/a
org.example.mbeans.MBeanRegistry\$Proxy\$_\$\$_WeldClientProxy	1 (0 %)	48 B (0 %)	n/a
org.example.mbeans.AreaCalculator	2 (0 %)	48 B (0 %)	n/a
org.example.beans.UserBean\$Proxy\$_\$\$_WeldClientProxy	1 (0 %)	40 B (0 %)	n/a
org.example.beans.PointBean	1 (0 %)	40 B (0 %)	n/a
org.example.beans.ResultsBean\$Proxy\$_\$\$_WeldClientProxy	1 (0 %)	32 B (0 %)	n/a
org.example.mbeans.MBeanRegistry	1 (0 %)	32 B (0 %)	n/a
org.example.service.DatabaseService	1 (0 %)	32 B (0 %)	n/a
org.example.entities.UserEntity	1 (0 %)	24 B (0 %)	n/a
org.example.beans.UserBean	1 (0 %)	24 B (0 %)	n/a
org.example.beans.ClockBean	1 (0 %)	24 B (0 %)	n/a
org.example.beans.ResultsBean	1 (0 %)	16 B (0 %)	n/a
org.example.mbeans.interfaces.AreaCalculatorMBean	0 (0 %)	0 B (0 %)	n/a
org.example.mbeans.interfaces.PointStatisticsMBean	0 (0 %)	0 B (0 %)	n/a
org.example.entities.PointEntity	0 (0 %)	0 B (0 %)	n/a
org.example.validation.RValidator	0 (0 %)	0 B (0 %)	n/a
org.example.validation.YValidator	0 (0 %)	0 B (0 %)	n/a
org.example.validation.XValidator	0 (0 %)	0 B (0 %)	n/a

Рис. 3.5 – распределение памяти среди пользовательских классов

По данным Memory Sampler VisualVM наибольший объем памяти среди всех классов занимают объекты класса `byte[]`: 29 761 008 В, 302 135 объектов.

Для определения владельцев этих объектов был построен Heap Dump. Анализ показал, что крупнейшие `byte[]` удерживаются объектами `java.util.zip.ZipFile$Source`. Следовательно, эти объекты относятся к инфраструктуре JVM/WildFly и загруженным JAR/ZIP-ресурсам, а не к пользовательским классам приложения.

Пользовательского класса `org.example.*`, в экземплярах которого находились бы крупнейшие `byte[]`, в Heap Dump обнаружено не было. Среди пользовательских классов приложения наибольший собственный размер имеет `org.example.mbeans.PointStatistics`: 2 объекта, 80 В.

Выполнение задания №4

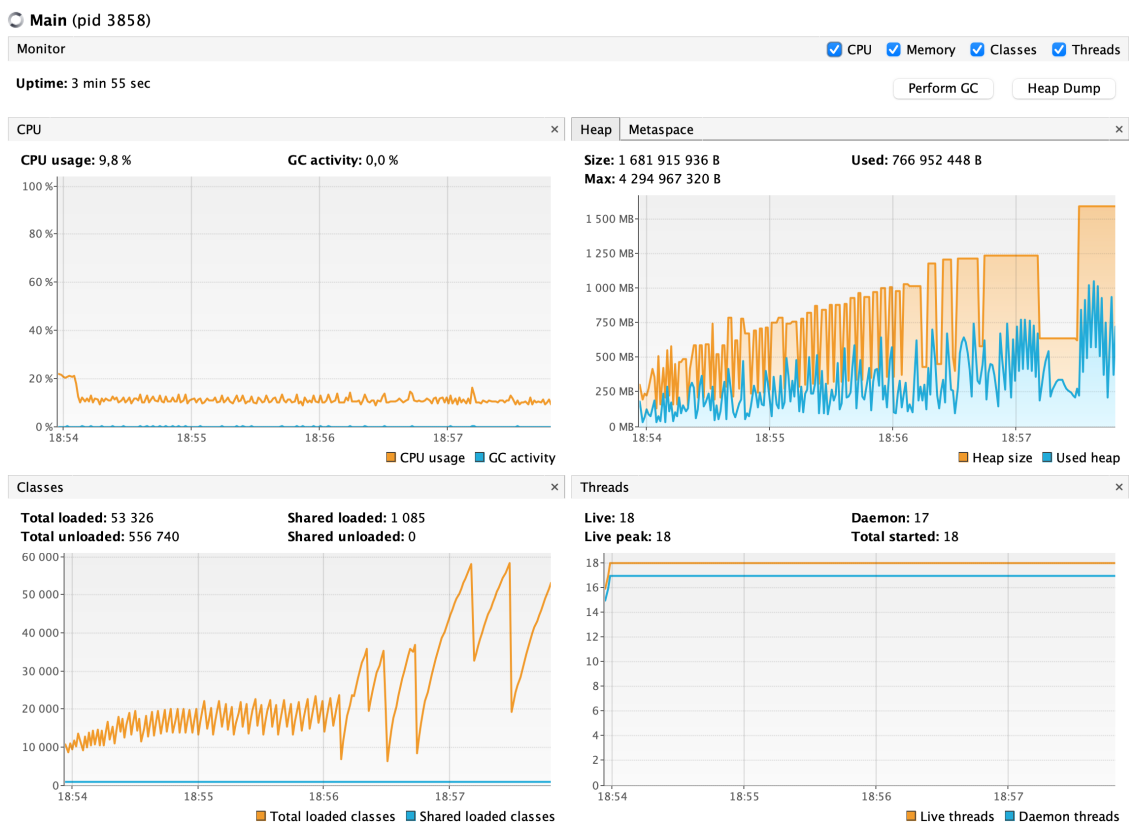


Рис. 4.1 – показатели до исправления

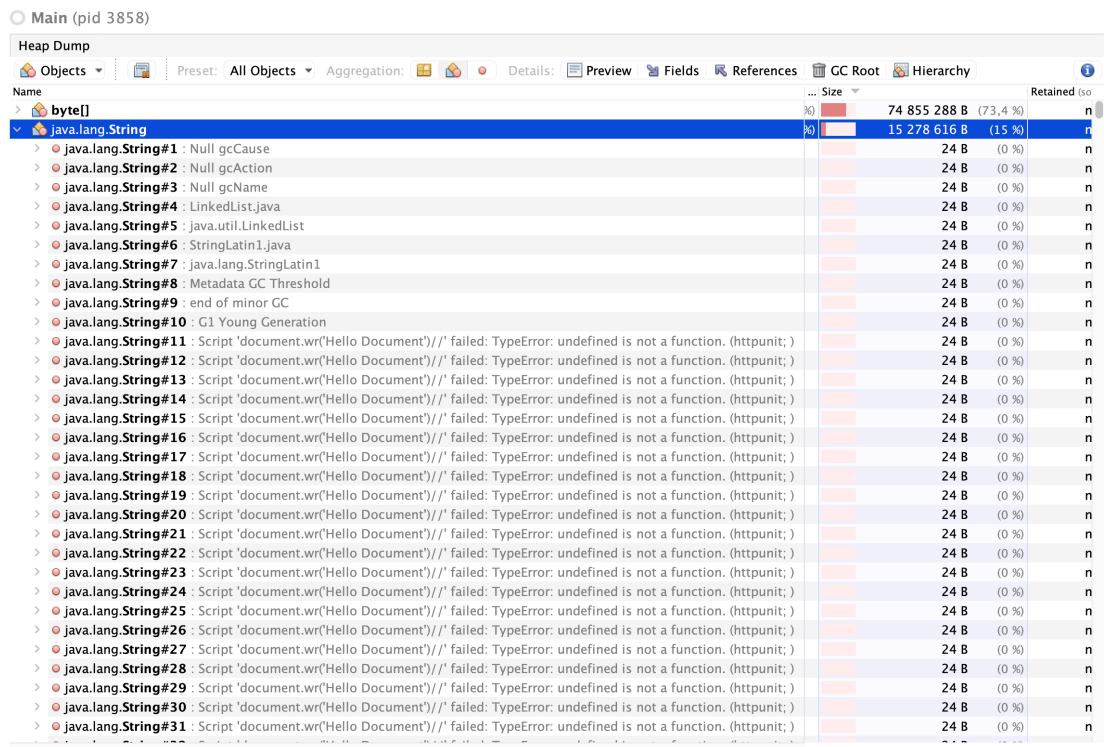


Рис. 4.2 – Heap Dump до исправления

Method	Execution time(ms)
100.0% Main.main(String[]) +82 calls	44,368
96.9% com.meterware.httpunit.WebWindow.updateWindow(String, WebResponse, RequestContext) +11 calls	43,010
89.4% com.meterware.httpunit.WebResponse.getFrameRequests() +36 calls	39,681
86.6% org.apache.xerces.parsers.DOMParser.parse(InputSource) +439 library calls	38,425
< 1% com.meterware.httpunit.parsing.ScriptFilter.characters(XMLString, Augmentations)	20
< 1% com.meterware.httpunit.parsing.ScriptFilter.characters(XMLString, Augmentations)	30
73.3% com.meterware.httpunit.parsing.ScriptFilter.endElement(QName, Augmentations) +1,125 calls	32,528
< 1% com.meterware.httpunit.parsing.ScriptFilter.endElement(QName, Augmentations)	20
< 1% com.meterware.httpunit.parsing.ScriptFilter.startElement(QName, XMLAttributes, Augmentations)	211
< 1% com.meterware.httpunit.parsing.ScriptFilter.endElement(QName, Augmentations)	20
2.5% com.meterware.httpunit.parsing.NekoDOMParser.newParser(DocumentAdapter, URL)	1,126
7.4% com.meterware.httpunit.javascript.JavaScriptEngineFactory.associate(WebResponse)	3,299
< 1% com.meterware.httpunit.HttpUnitOptions.getScriptingEngine()	10
2.2% com.meterware.httpunit.WebWindow.getResource(WebRequest)	983
< 1% java.lang.Thread.run() +3 library calls	10

Рис. 4.3 – Call Tree

Method	Execution time(ms)
99.6% com.meterware.httpunit.parsing.ScriptFilter.getTranslatedScript(String, String)	32,386
99.6% com.meterware.httpunit.scripting.ScriptableDelegate.runScript(String, String)	32,386
99.6% com.meterware.httpunit.javascript.JavaScript\$Window.executeScript(String, String)	32,386
99.6% com.meterware.httpunit.javascript.JavaScript\$JavaScriptEngine.executeScript(String, String)	32,386
99.0% org.mozilla.javascript.Context.evaluateString(Scriptable, String, String, int, Object)	32,206
99.0% org.mozilla.javascript.Context.evaluateReader(Scriptable, Reader, String, int, Object)	32,206
57.8% org.mozilla.javascript.Context.compileReader(Scriptable, Reader, String, int, Object)	18,806
38.7% stale_methodID	12,590

Рис. 4.4 – Call Tree

Локализация:

Программа была проанализирована через VisualVM. На графике Heap было видно, что используемая память постоянно растет, а размер heap доходил примерно до 1.68 GB. На графике Classes также наблюдался резкий рост количества загруженных классов. Для программы, которая многократно выполняет один и тот же запрос к сервлету, такое поведение выглядит аномально.

Для уточнения причины был использован Call Tree профилировщика IntelliJ IDEA. Основная цепочка выполнения проходила не через пользовательский код, а через HttpUnit:

```
WebResponse.getFrameRequests()
```

```
-> DOMParser.parse(...)
```

```
-> ScriptFilter.getTranslatedScript(...)
```

```
-> Context.evaluateString(...)
```

Это показало, что основная нагрузка связана с обработкой JavaScript внутри HTML-ответа. В коде сервлета был найден ошибочный JavaScript:

```
document.wr('Hello Document')
```

Такого метода не существует, поэтому HttpUnit получал JavaScript-ошибку при каждом запросе. При этом в Main.java включена настройка:

```
HttpUnitOptions.setExceptionsThrownOnScriptError(false);
```

Из-за нее программа не завершалась с исключением, а HttpUnit сохранял сообщения об ошибках во внутреннем списке.

Гипотеза подтвердилась через Heap Dump: среди объектов java.lang.String было найдено много одинаковых строк вида:

```
Script 'document.wr(...)' failed: TypeError ...
```

В References было видно, что эти строки удерживаются через java.util.ArrayList.elementData, а сам список находится в статическом поле:

```
com.meterware.httpunit.javascript.JavaScript._errorMessages
```

Исправление:

Исправление было выполнено без изменения сервлета и без модификации исходного кода библиотеки HttpUnit. Для этого использован штатный API HttpUnit, который позволяет получить и очистить накопленные сообщения JavaScript-ошибок.

В основной цикл программы добавлено ограничение на количество хранимых ошибок:

```
final int maxStoredScriptErrors = 50;
```

После каждого запроса программа проверяет, сколько сообщений уже накоплено во внутреннем списке HttpUnit:

```
if (HttpUnitOptions.getScriptErrorMessages().length >= maxStoredScriptErrors) {  
    HttpUnitOptions.clearScriptErrorMessages();  
}
```

Если количество сообщений достигает 50, вызывается:

```
HttpUnitOptions.clearScriptErrorMessages();
```

Этот метод очищает внутренний список ошибок HttpUnit. За счет этого сообщения о повторяющейся JavaScript-ошибке больше не могут накапливаться бесконечно.

После исправления программа по-прежнему воспроизводит исходный сценарий: сервлет возвращает тот же HTML с ошибочным JavaScript, а HttpUnit продолжает обрабатывать ответ. Однако количество сохраненных сообщений об ошибках теперь ограничено.

Также, чтобы ограничить рост классов и Heap, был использован флаг -Xmx29m, ограничивающий размер кучи сверху до 29мб.

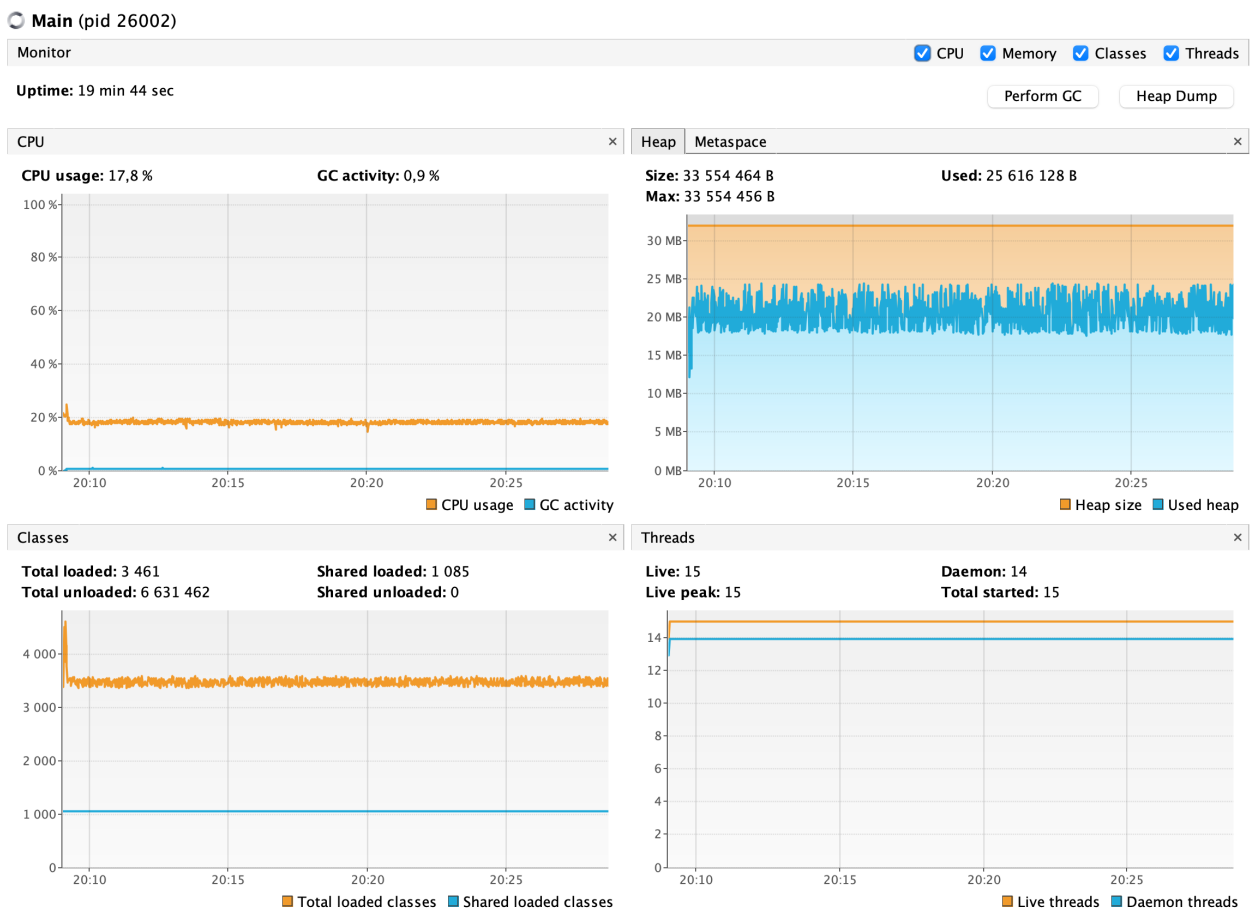


Рис. 4.5 – показатели после исправления

Main (pid 26002)

Heap Dump

Objects Preset: All Objects Aggregation: Details: Preview Fields References GC Root Hierarchy

Name	Count	Size	Retained (sort to get)
> java.util.TreeMap\$Entry	107 196 (23,8 %)	4 287 840 B (27,9 %)	n/a
> java.util.TreeMap	34 036 (7,5 %)	1 633 728 B (10,6 %)	n/a
> byte[]	26 877 (6 %)	1 533 528 B (10 %)	n/a
> java.lang.Long	54 448 (12,1 %)	1 306 752 B (8,5 %)	n/a
> javax.management.openmbean.CompositeDataSupport	34 000 (7,5 %)	816 000 B (5,3 %)	n/a
> java.util.LinkedHashMap\$Entry	16 965 (3,8 %)	678 600 B (4,4 %)	n/a
> java.lang.Object[]	19 982 (4,4 %)	647 744 B (4,2 %)	n/a
✓ java.lang.String	25 930 (5,7 %)	622 320 B (4,1 %)	n/a
> java.lang.String#1 : http://test.meterware.com/myServlet		24 B (0 %)	n/a
> java.lang.String#2 : Main		24 B (0 %)	n/a
> java.lang.String#3 : Main		24 B (0 %)	n/a
> java.lang.String#4 [GC root - JNI global] : -2147483648		24 B (0 %)	n/a
> java.lang.String#5 [GC root - JNI global] : <null_sentinel>		24 B (0 %)	n/a
> java.lang.String#6 [GC root - JNI global] : Delayed StackOverflowError due to ReservedStack		24 B (0 %)	n/a
> java.lang.String#7 : xmlns		24 B (0 %)	n/a
> java.lang.String#8 : CDATA		24 B (0 %)	n/a
> java.lang.String#9 : XMLNS:		24 B (0 %)	n/a
> java.lang.String#10 : XMLNS		24 B (0 %)	n/a
> java.lang.String#11 : org.mozilla.javascript.gen		24 B (0 %)	n/a
> java.lang.String#12 : httpunit		24 B (0 %)	n/a
> java.lang.String#13 : com.intellij.rt.execution.application.AppMainV2\$1		24 B (0 %)	n/a
> java.lang.String#14 : com.intellij.rt.execution.application		24 B (0 %)	n/a
> java.lang.String#15 : /Applications/IntelliJ%20IDEA.app/Contents/lib/idea_rt.jar		24 B (0 %)	n/a
> java.lang.String#16 : file:///Applications/IntelliJ%20IDEA.app/Contents/lib/idea_rt.jar		24 B (0 %)	n/a
> java.lang.String#17 : (Ljava/lang/Object;Ljava/lang/Object;Ljava/lang/Object;)Ljava/lang/O		24 B (0 %)	n/a
> java.lang.String#18 : L		24 B (0 %)	n/a
> java.lang.String#19 : V		24 B (0 %)	n/a
> java.lang.String#20 : /jdk.jdwp.agent		24 B (0 %)	n/a
> java.lang.String#21 : jrt:/jdk.jdwp.agent		24 B (0 %)	n/a
> java.lang.String#22 : /java.base		24 B (0 %)	n/a
> java.lang.String#23 : jrt:/java.base		24 B (0 %)	n/a
> java.lang.String#24 : (Ljava/lang/Object;Ljava/lang/Object;)V		24 B (0 %)	n/a
> java.lang.String#25 : (Ljava/lang/Object;Ljava/lang/invoke/MemberName;)V		24 B (0 %)	n/a

Рис. 4.6 – Heap Dump после исправления

Итог:

После добавления периодической очистки списка JavaScript-ошибок программа была повторно запущена без изменения сервлета и без исправления самого ошибочного JavaScript. На мониторинге VisualVM видно, что heap-память не растёт, как и диаграмма загруженных классов.

Также изменился Heap Dump. До исправления среди объектов `java.lang.String` было видно большое количество одинаковых строк с сообщением:

```
Script 'document.wr('Hello Document')//' failed: TypeError ...
```

После исправления таких повторяющихся строк в начале списка `java.lang.String` больше нет. Вместо них отображаются обычные служебные строки JVM и VisualVM, связанные с GC и диагностикой. Это подтверждает, что сообщения JavaScript-ошибок больше не накапливаются в памяти в неограниченном количестве.

Программа

Исходный код проекта лежит в публичном GitHub-репозитории по ссылке:

https://github.com/vladlenblch/ITMO_VT/tree/main/2_course/cse/lab4

Вывод:

Во время выполнения данной лабораторной работы я:

- 1) Повторил навыки профилирования
- 2) Повторил навыки поиска багов и их исправления
- 3) Научился работать с JConsole
- 4) Научился работать с VisualVM
- 5) Научился работать с профилировщиком IntelliJ Idea